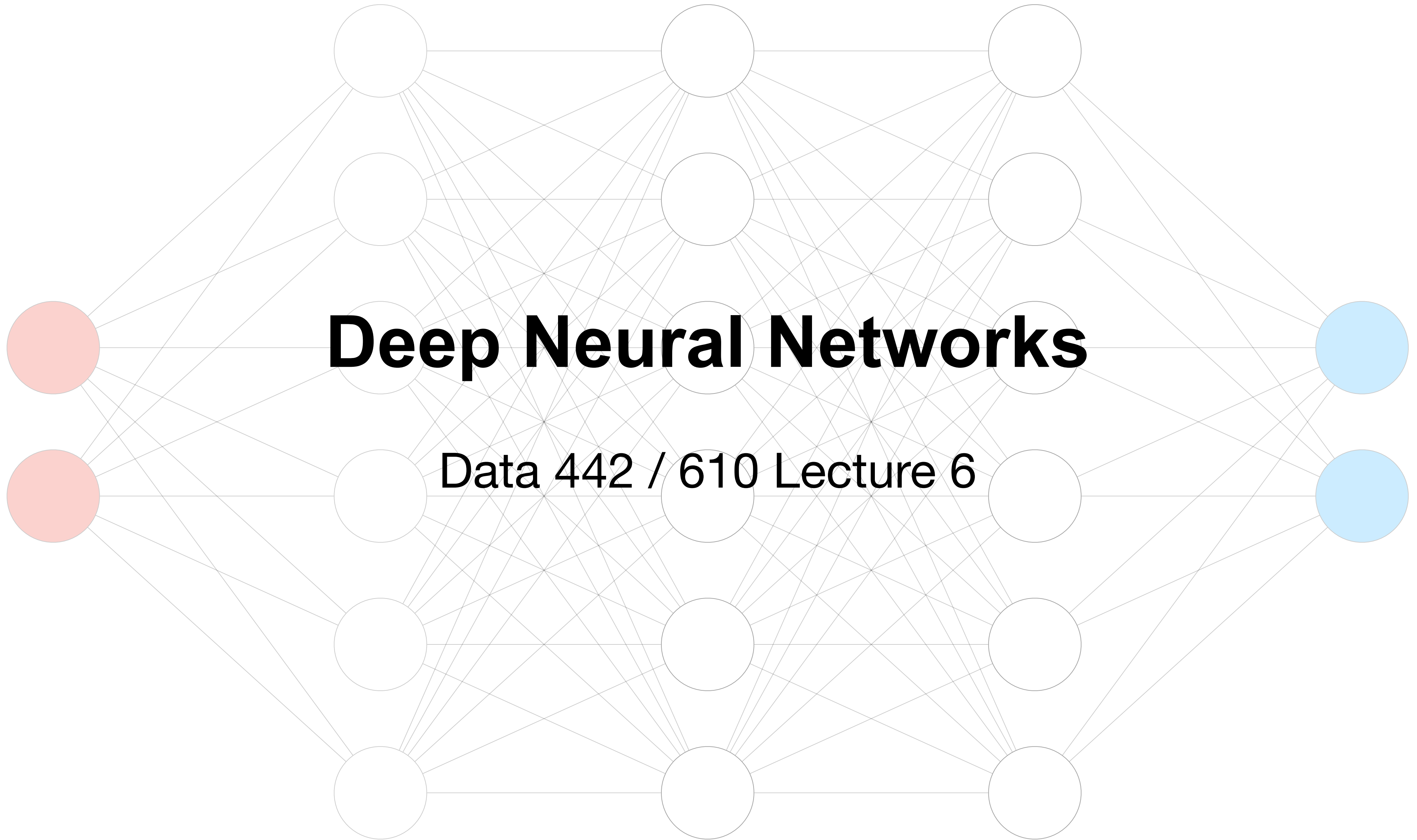
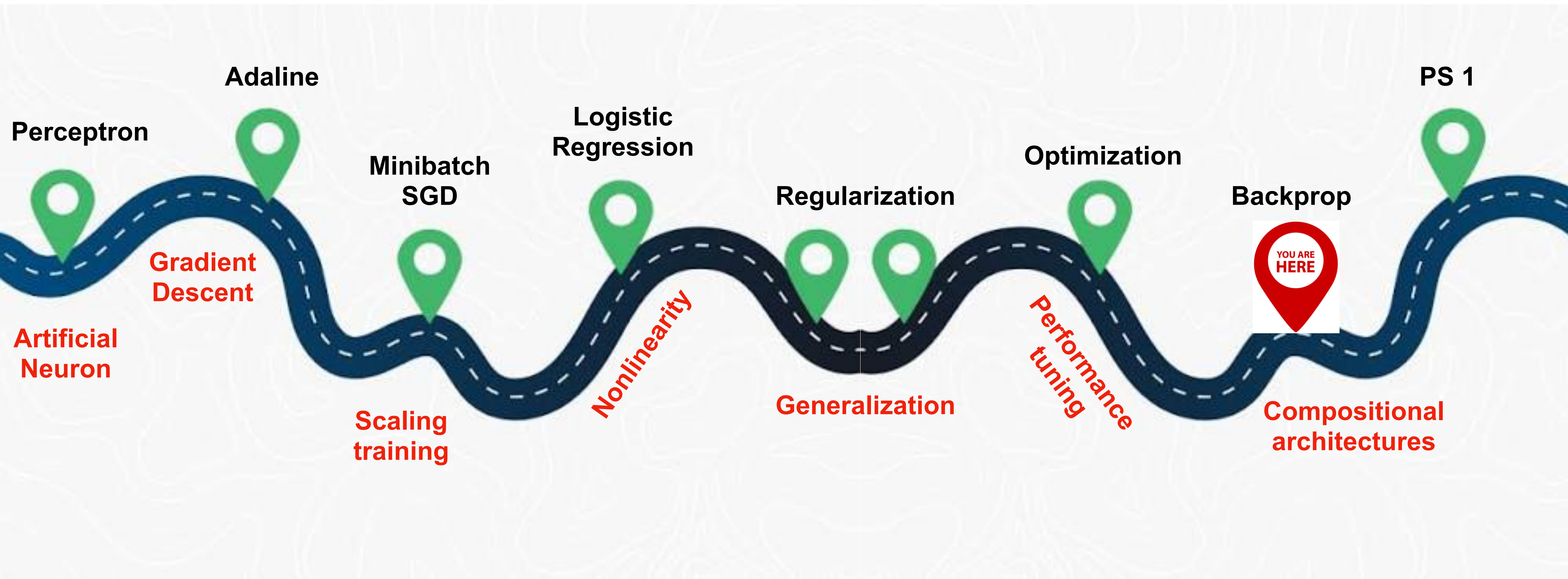


Deep Neural Networks

Data 442 / 610 Lecture 6



Roadmap to Deep Learning



Outline

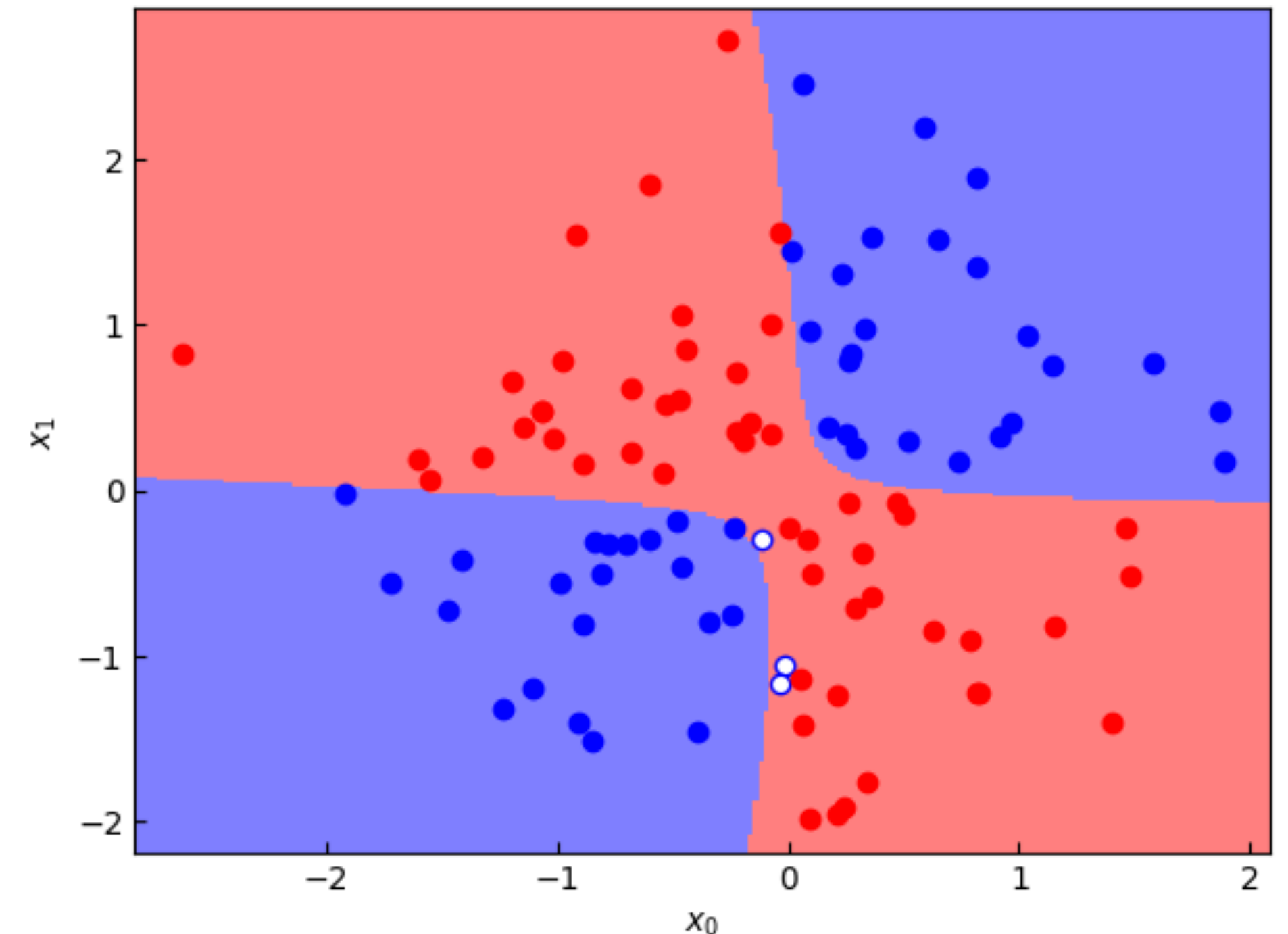
- Review of nonlinear optimization, kernel trick
- Deep neural networks
- Backpropagation

Relevant Reading: *Deep Learning*, Chapter 6.5

Make up OH: Wednesday, Sep 23, 9:30-11:30 am

Nonlinear optimization

- Optimizing nonlinear functions is challenging because optimization methods for linear functions may no longer apply
- One way around this is to do *linear* optimization on a nonlinear transformation of the inputs $z = \phi(x)$ (polynomial LR)
- Another way is to exploit the **kernel trick**



$$f(x) = b + \sum_i \alpha_i \phi(x)^\top \phi(x^i)$$



$$f(x) = b + \sum_i \alpha_i k_\phi(x, x^i)$$

Nonlinear optimization

Without knowing the “right” feature function $\phi(x)$, the **kernel trick** lets us *implicitly* compute dot products between high-(or infinite-)dimensional feature vectors

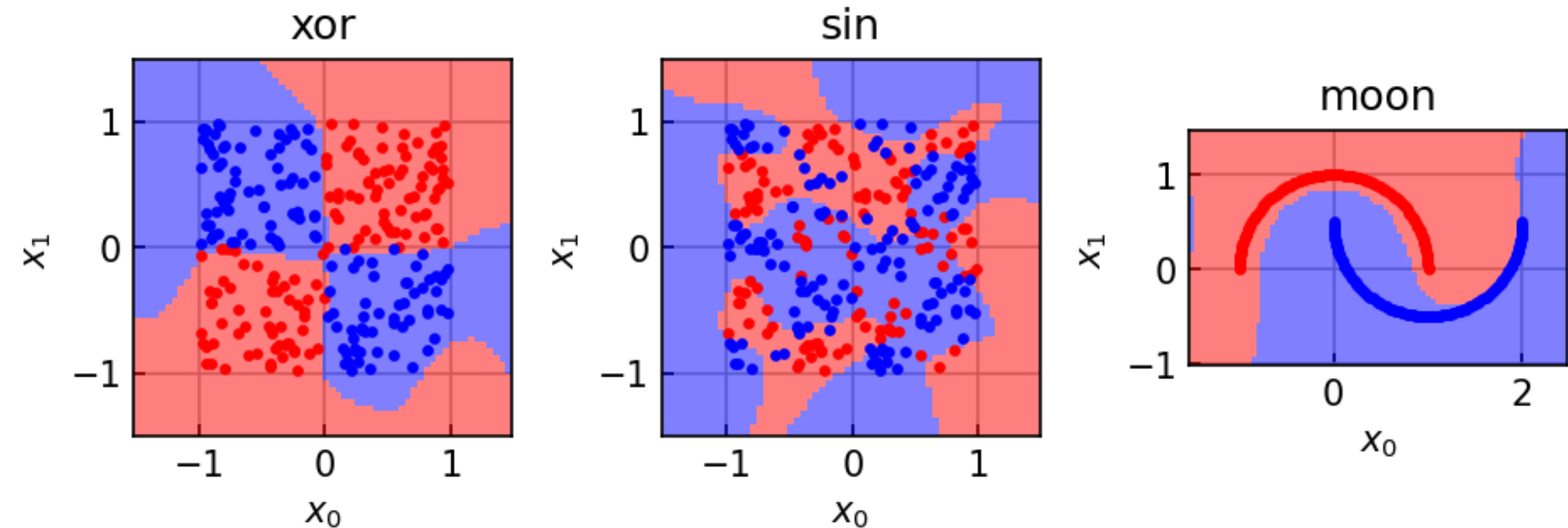
Drawback #1: Training requires inverting an $N \times N$ matrix [$O(N^3)$]

$$\vec{\alpha} = (K + \lambda I)^{-1} \hat{Y}$$

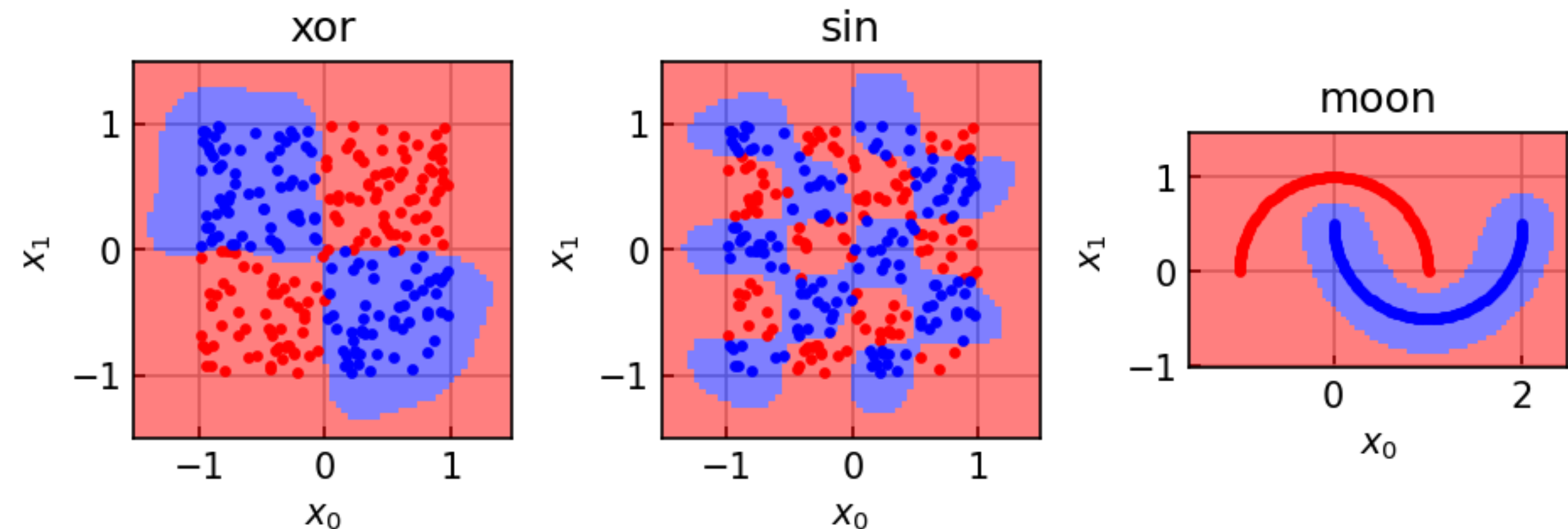
Drawback #2: Inference requires N kernel function evaluations

$$y(x) = \sum_{i=1}^N \alpha_i k(x_i, x)$$

Degree-10 Polynomial LR



Kernel Ridge Regression



Nonlinear optimization

Degree-10 Polynomial LR

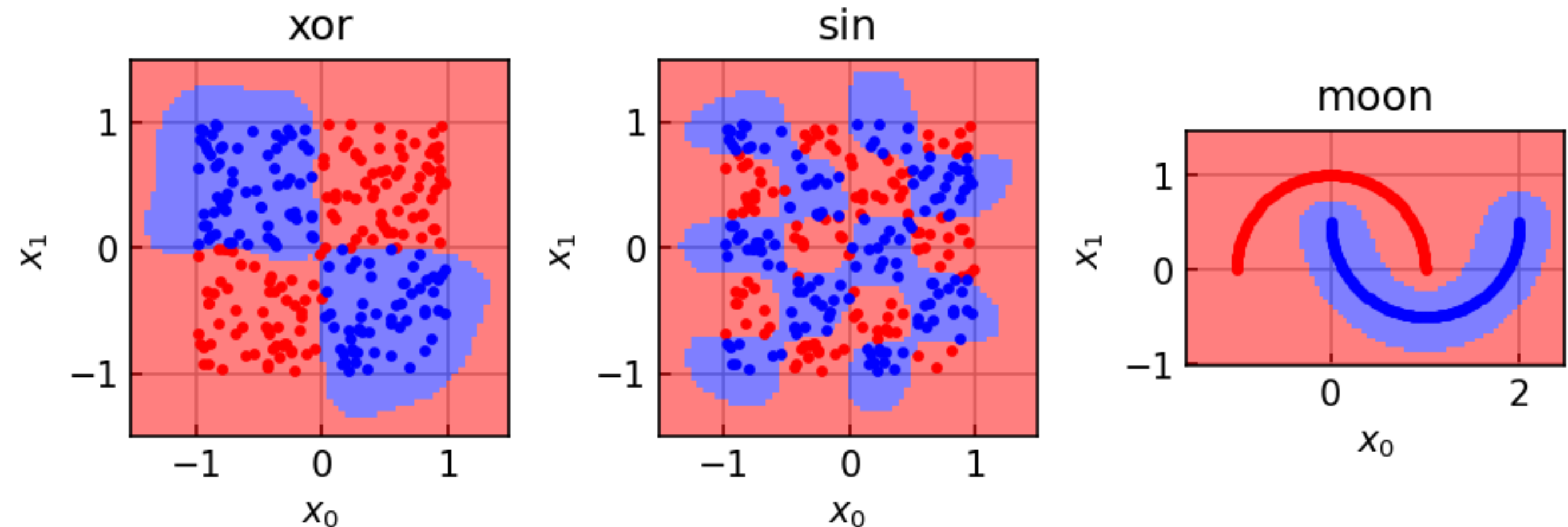
But do I always need an infinite-dimensional feature vector $\phi(x)$?

$$\vec{\alpha} = (K + \lambda I)^{-1} \hat{Y}$$

Drawback 2: Inference requires N kernel function evaluations

$$y(x) = \sum_{i=1}^N \alpha_i k(x_i, x)$$

Kernel Ridge Regression



Nonlinear optimization

Degree-10 Polynomial LR

But do I always need an infinite-dimensional feature vector $\phi(x)$?

$$\vec{\alpha} = (K + \lambda I)^{-1} \hat{Y}$$

Kernel Ridge Regression

Deep neural networks learn nonlinear features $\phi(x)$ from data

Roadmap for today

Build an MLP

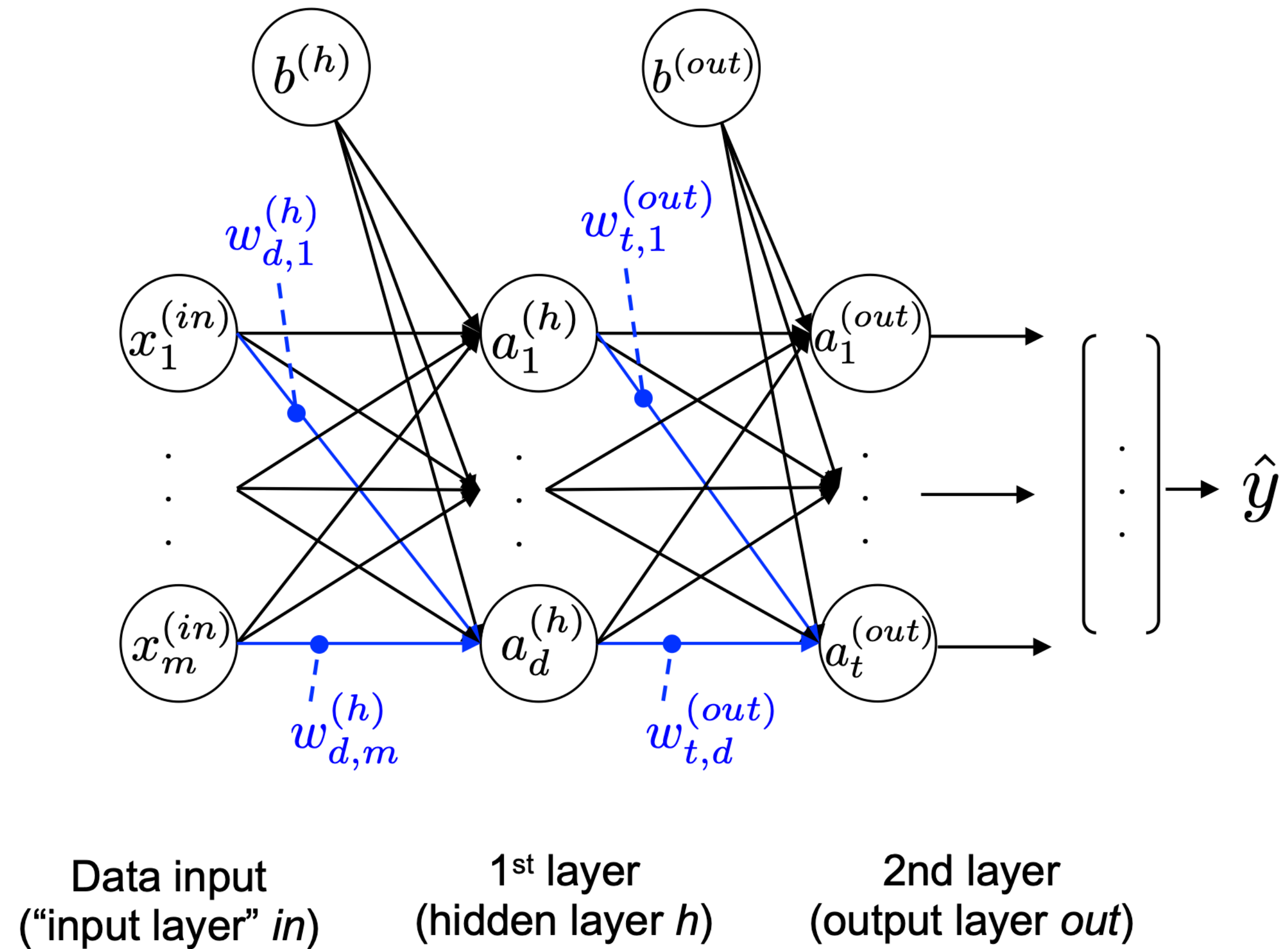
- Consider MNIST multi-class classification
- Code an 2-layer MLP with one hidden layer and one output layer
- Derive weight updates using MSE loss function
- Compute weight updates using back propagation
- Apply weight updates using mini-batch stochastic gradient descent

A harder problem: MNIST



<https://web.archive.org/web/20200430193701/http://yann.lecun.com/exdb/mnist/>

Multi-layer neural network



Courtesy of C. Fanelli

Gradient Descent with a MLP

Goal: Find $\mathbf{w}$ to minimize the loss $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y^i - y_{NN}^i)^2$

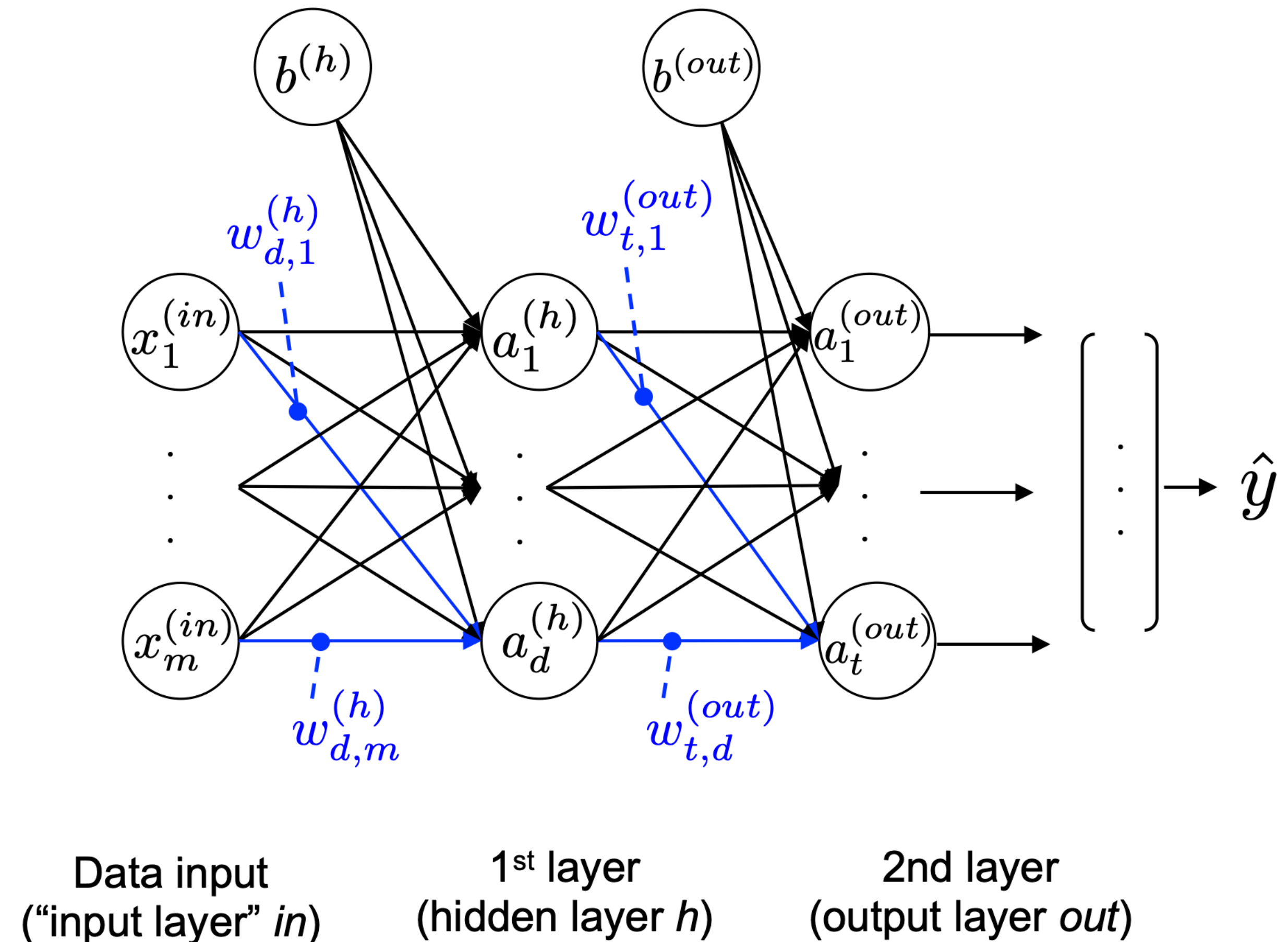
Algorithm: For each training example $\mathbf{x}^i$

1. Compute the output $y_{NN}^i = \sigma(\mathbf{w}_o^\top \sigma(\mathbf{w}_h^\top \mathbf{x}^i + \mathbf{b}_h) + \mathbf{b}_o)$
2. Update the weights and bias

$$w_j \leftarrow w_j + \Delta w_j$$

$$b \leftarrow b + \Delta b$$

How are we going to do this???



Courtesy of C. Fanelli

Gradient Descent with a MLP

Goal: Find $\mathbf{w}$ to minimize the loss $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y^i - y_{NN}^i)^2$

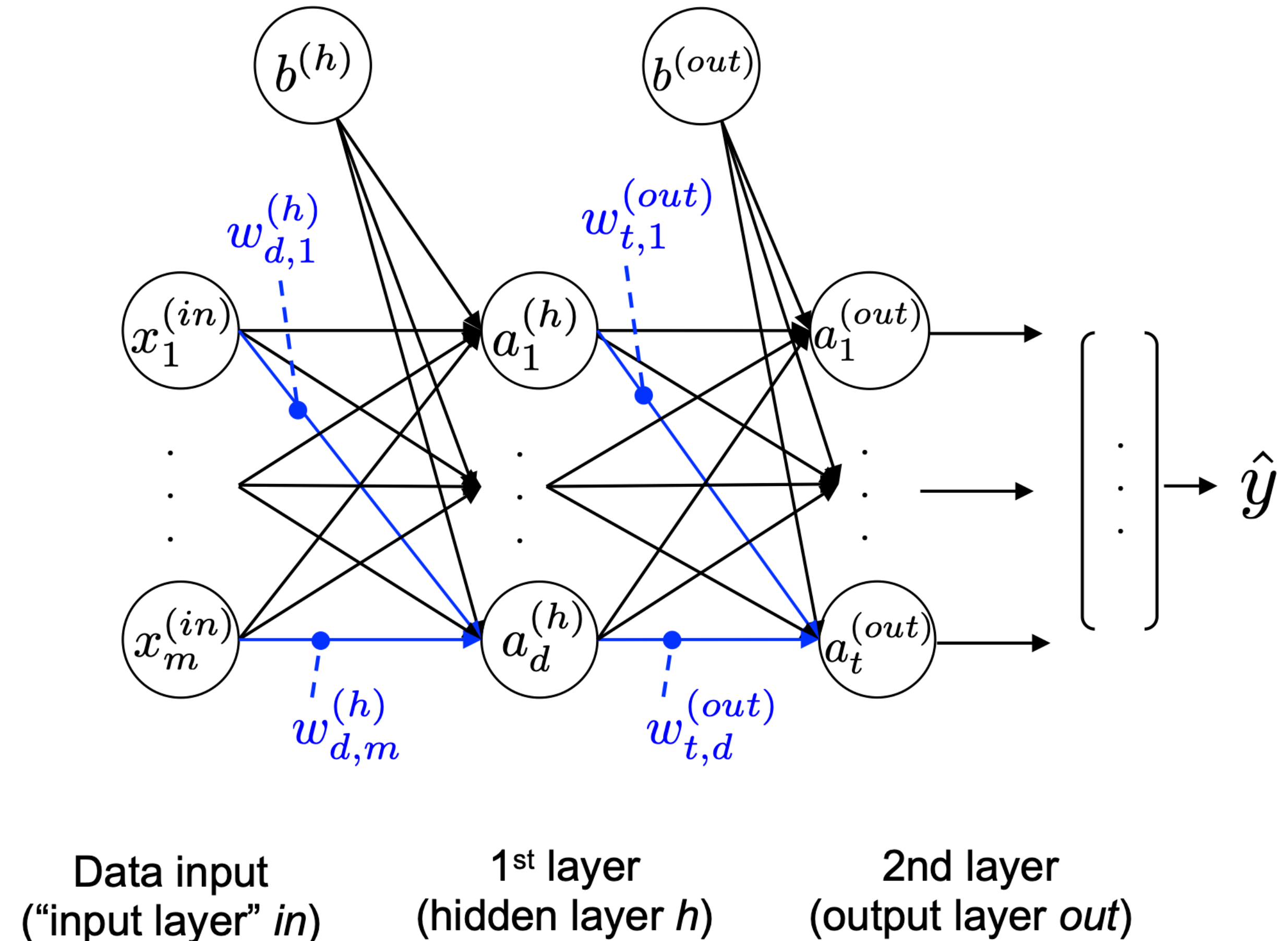
Algorithm: For each training example $\mathbf{x}^i$

1. Compute the output $y_{NN}^i = \sigma(\mathbf{w}_o^\top \sigma(\mathbf{w}_h^\top \mathbf{x}^i + \mathbf{b}_h) + \mathbf{b}_o)$
2. Update the weights and bias

$$w_j \leftarrow w_j + \Delta w_j$$

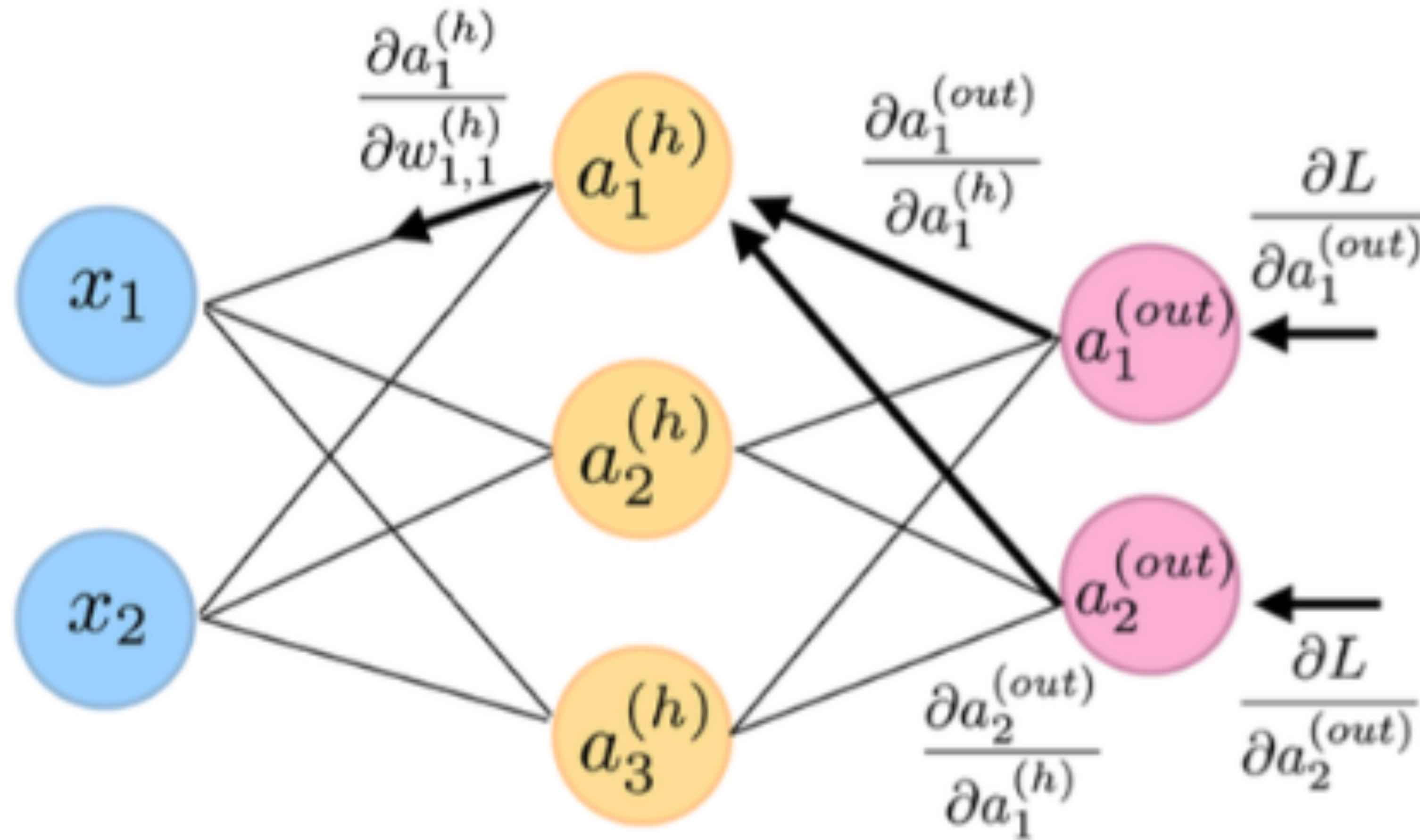
$$b \leftarrow b + \Delta b$$

$$\text{Chain rule: } \frac{d}{dx} f(g(x)) = \frac{df}{dg} \frac{dg}{dx}$$



Courtesy of C. Fanelli

Backpropagation



Backpropagation is an efficient way to compute gradients by proceeding layer-by-layer

Hands-on Demo: Building a MLP

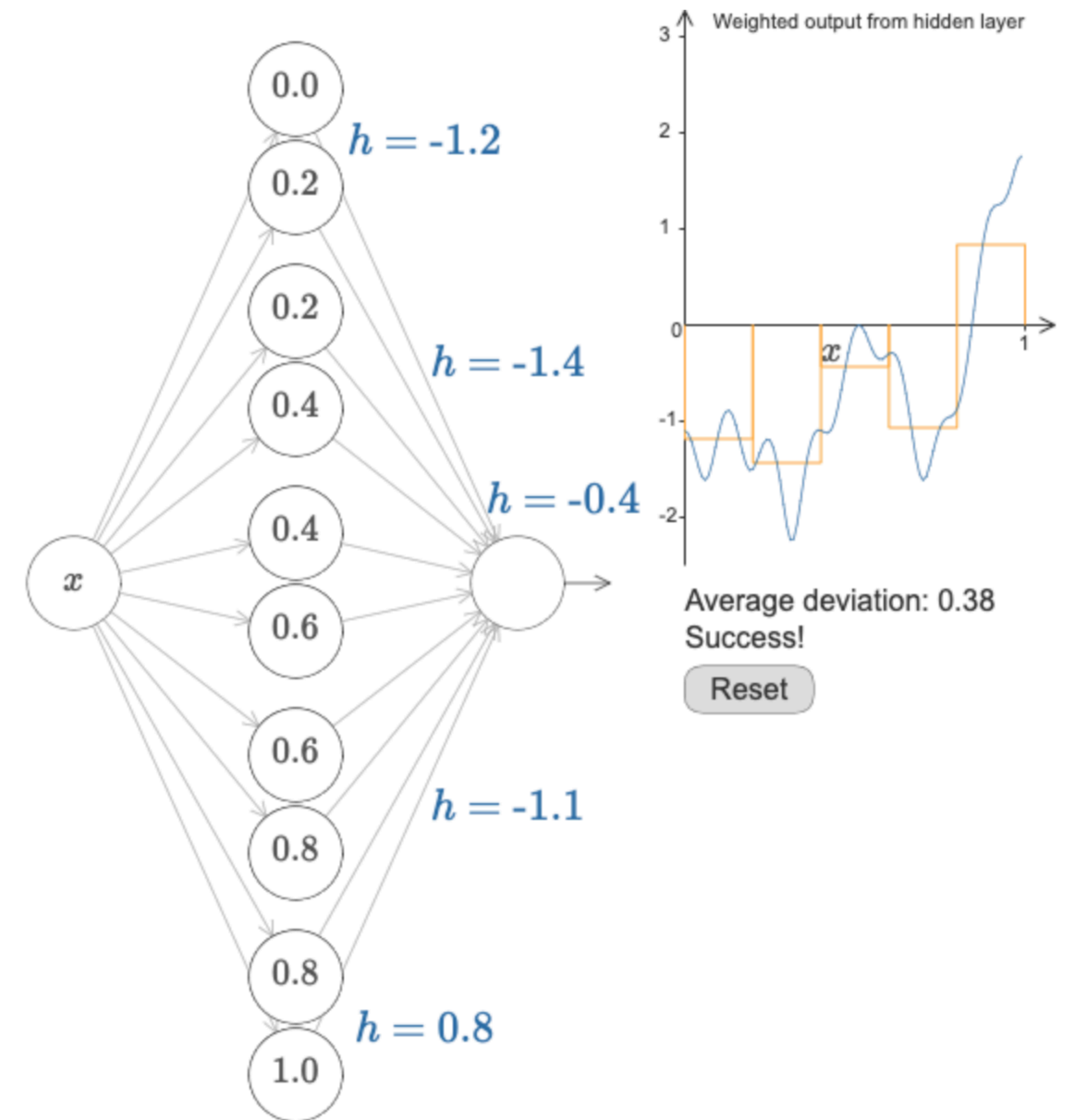


Notes on NN convergence

- Mini-batch SGD is an efficient and effective algorithm for training NNs
- It is believed that SGD avoids local minima through stochasticity
- It is believed that deep NNs may avoid local minima in general
- Hyperparameter tuning is crucial (and expensive) for optimal NN performance

Universal approximation theorem

- A feedforward neural network with at least one hidden layer and suitable activations can approximate an arbitrary continuous function to any degree of accuracy
- **Proof:** well beyond the scope of this class. But a nice explanation is given [here](#)
- In practice this means...very little



That's all for today!

- Slides will be posted on the course website: <https://colen-teaching.github.io/DATA442-Fall-2026>
- Lab demos will also be posted
- **PS1** due October 2
- Please make sure to install **pytorch** before next class